

LAPORAN PRAKTIKUM 4

INHERITANCE, GENERALIZATION & OVERRIDING

Identitas

Nama: Najwan Rachman
NIM: 251511024
Kelas:2A-D3
Mata Kuliah: Pemrograman Berorientasi Objek

Github: https://github.com/NajwanRachman/024_Najwan-Rachman_PBO04.git

PRAKTIKUM I — CIRCLE DAN CYLINDER

A.1 Circle

Pada percobaan pertama dibuat class Circle dengan dua atribut yaitu radius dan color. Atribut dibuat private sehingga aksesnya dilakukan melalui getter dan setter.

Class Circle memiliki beberapa constructor, yaitu constructor tanpa parameter, constructor dengan radius, dan constructor dengan radius serta color.

Method getArea() digunakan untuk menghitung luas lingkaran dengan rumus:

Luas = $\pi \times \text{radius}^2$

Method toString() digunakan untuk menampilkan informasi dari object Circle.

A.2 Cylinder

Class Cylinder dibuat sebagai turunan dari Circle menggunakan:

public class Cylinder extends Circle

Cylinder hanya menambahkan atribut height, sedangkan radius dan color tidak dibuat ulang karena sudah dimiliki oleh Circle.

Constructor Cylinder menggunakan super() atau super(radius) untuk memanggil constructor dari Circle.

Method yang digunakan pada Cylinder yaitu:

1.getHeight()

2.setHeight()

3.getVolume()

4.getArea()

5.toString()

A.3 Pengujian Cylinder

Pengujian dilakukan dengan membuat tiga object:

Cylinder c1 = new Cylinder();

Cylinder c2 = new Cylinder(10.0);

Cylinder c3 = new Cylinder(2.0, 10.0);

Hasil yang diperoleh:

c1

radius = 1.0

height = 1.0

base area = 3.141592653589793

volume = 3.141592653589793

c2

radius = 1.0

height = 10.0

base area = 3.141592653589793

volume = 31.41592653589793

c3

radius = 2.0

height = 10.0

base area = 12.566370614359172

volume = 125.66370614359172

A.4 Constructor Chain

Untuk melihat urutan constructor, ditambahkan output sementara pada constructor Circle dan Cylinder.

Saat menjalankan:

new Cylinder(2.0, 10.0);

hasilnya:

Circle(double) dipanggil

Cylinder(double,double) dipanggil

Artinya constructor Circle dipanggil terlebih dahulu, kemudian constructor Cylinder.

Saat super(radius) dihapus, constructor Circle(double) tidak lagi dipanggil. Setelah percobaan selesai, super(radius) dikembalikan seperti semula

```
PS C:\Semester 3\PBO\Praktikum 4> javac -d . Circle.java Cylinder.java TestCylinder.java
PS C:\Semester 3\PBO\Praktikum 4> java TestCylinder
Circle(double) dipanggil
Cylinder(double,double) dipanggil
```

A.5 Encapsulation

Pada percobaan ini dicoba mengakses radius secara langsung dari class Cylinder.

Hasilnya muncul error:The field Circle.radius is not visible

Hal ini terjadi karena radius pada Circle menggunakan access modifier private.

Untuk mendapatkan nilai radius, digunakan:

getRadius()

Jadi subclass tetap bisa mendapatkan nilai radius tanpa mengakses atribut private secara langsung.

```
public double cobaAksesRadius() {
return radius;    The field Circle.radius is not visible
}
```

A.6 Overriding getArea()

Pada bagian ini dilakukan overriding terhadap method getArea() pada Cylinder. Setelah method tersebut dioverride, getVolume() yang menggunakan getArea() akan memanggil method milik Cylinder, sehingga hasil volume menjadi tidak sesuai.

Sebelum menggunakan super.getArea(), hasil volume adalah 1507.9644737231006.

Untuk memperbaikinya, getVolume() diubah menjadi menggunakan super.getArea(). Dengan begitu, perhitungan volume menggunakan luas alas dari Circle. Setelah diperbaiki, hasil volume menjadi 125.66370614359172.

Percobaan ini menunjukkan bahwa super.method() dapat digunakan untuk memanggil method superclass ketika method tersebut sudah dioverride oleh subclass.

Sebelum:

```
c3
radius = 2.0
height = 10.0
base area = 150.79644737231007
volume = 1507.9644737231006
```

Sesudah:

```
c3
radius = 2.0
height = 10.0
base area = 150.79644737231007
volume = 125.66370614359172
```

A.7 Overriding toString() dan @Override

Method toString() pada Cylinder dioverride agar informasi height ikut ditampilkan.

```
@Override
public String toString() {
    return "Cylinder: subclass of " + super.toString()
          + ", height=" + height;
}
```

Pada method tersebut digunakan super.toString() supaya informasi dari Circle tetap digunakan.

Kemudian dilakukan percobaan dengan mengubah nama method menjadi toStrng() tetapi annotation @Override tetap digunakan. Hasilnya compiler memberikan error karena method tersebut tidak melakukan overriding terhadap method pada superclass.

```
PS C:\Semester 3\PBO\Praktikum 4> javac -d . Circle.java Cylinder.java TestCylinder.java
Cylinder.java:38: error: toStrng() in Cylinder does not override or implement a method from a supertype
    @Override
    ^
1 error
```

B. PRAKTIKUM II — SHAPE HIERARCHY

B.1 Shape

Shape digunakan sebagai superclass yang memiliki atribut:

color

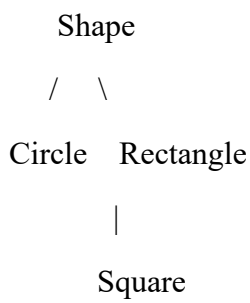
filled

Keduanya dibuat private.

Class ini memiliki constructor, getter, setter, dan toString().

B.2 Struktur Inheritance

Hierarchy yang dibuat:



Circle dan Rectangle merupakan subclass dari Shape, sedangkan Square merupakan subclass dari Rectangle.

B.3 Circle

Circle mewarisi atribut dan behavior dari Shape, kemudian menambahkan atribut radius.

Method yang digunakan:

getArea()

getPerimeter()

Constructor Circle menggunakan super() untuk memanggil constructor Shape.

B.4 Rectangle

Rectangle merupakan subclass dari Shape.

Atribut khususnya:

width

length

Method yang digunakan:

getArea()

getPerimeter()

Constructor juga menggunakan super() untuk mengisi bagian object yang berasal dari Shape.

B.5 Square

Square merupakan subclass dari Rectangle.

Square tidak membuat atribut baru karena width dan length sudah diwarisi dari Rectangle.

Saat membuat object Square, kedua nilai tersebut dibuat sama:

super(side, side);

Jadi aturan dasarnya adalah:

width = length

B.6 Masalah Setter pada Square

Sebelum setter dioverride, dilakukan pengujian:

Square s = new Square(5.0);

s.setWidth(8);

Hasil:

Sebelum:

width = 5.0

length = 5.0

Sesudah setWidth(8):

width = 8.0

length = 5.0

Hasil tersebut menunjukkan masalah karena sebuah Square seharusnya mempunyai width dan length yang sama.

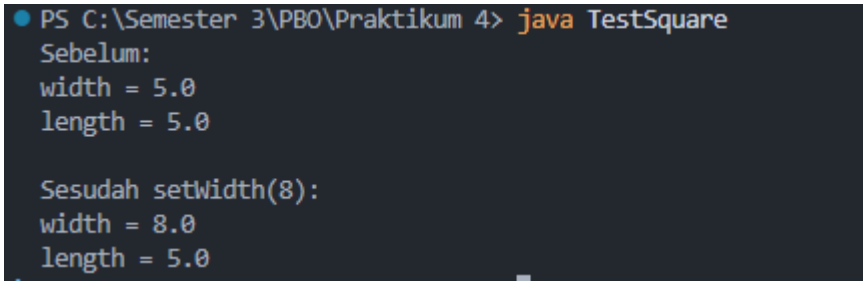
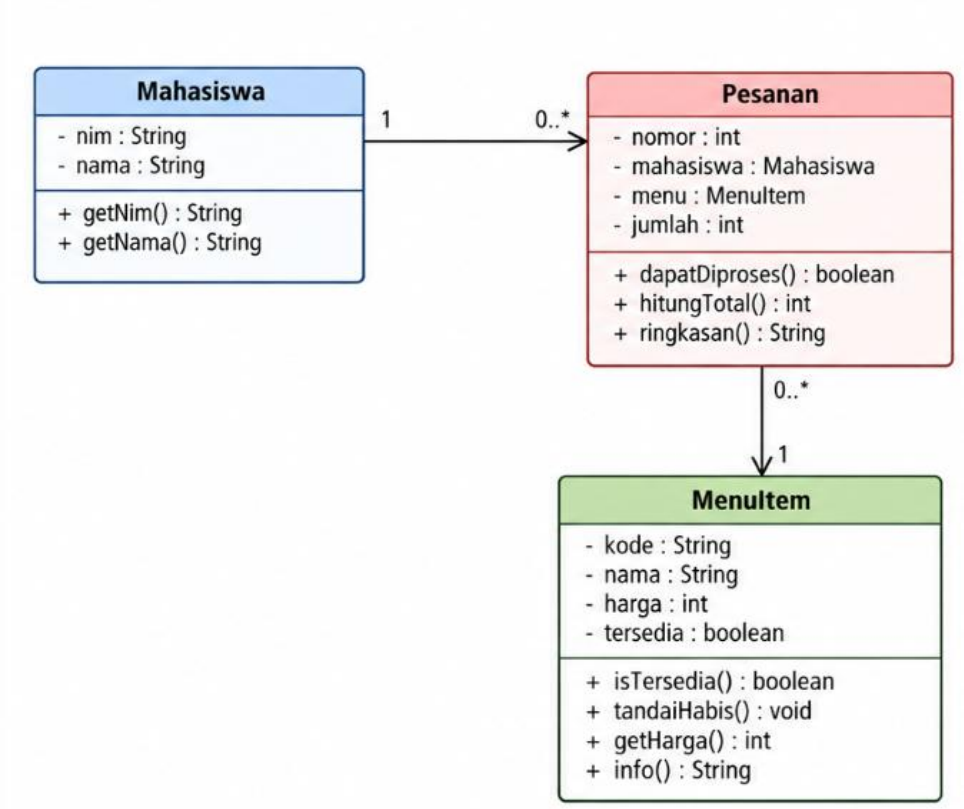


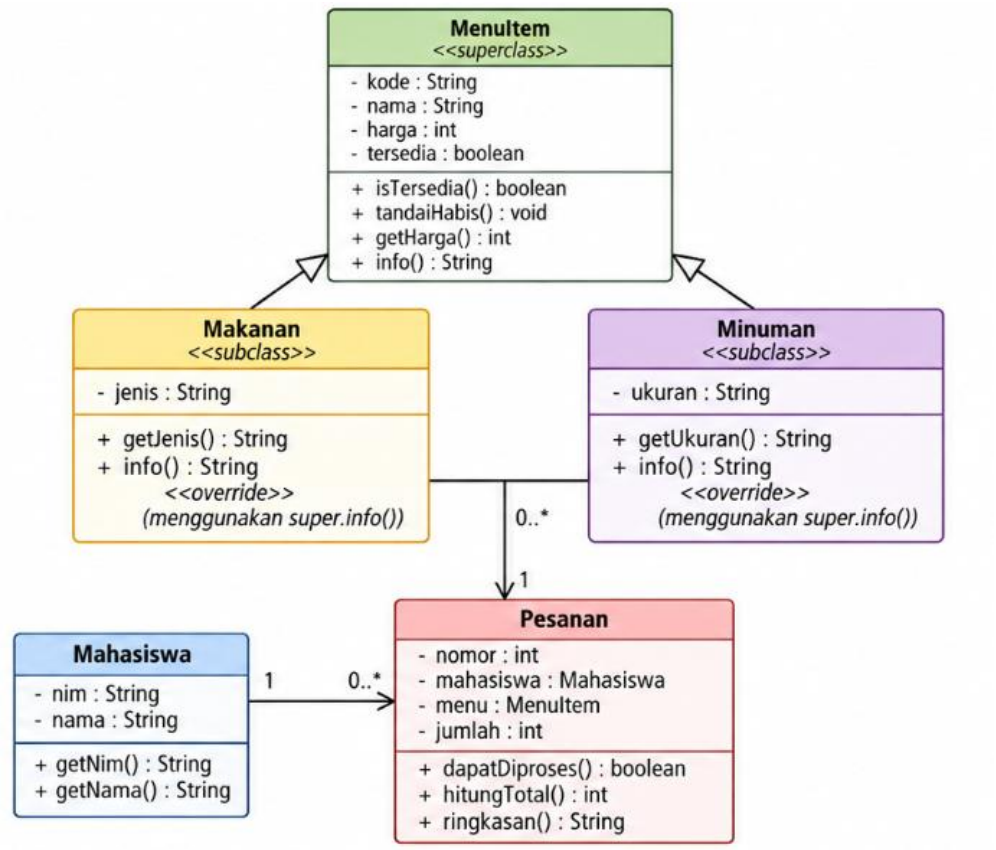
Diagram sebelum



Keterangan:

- Mahasiswa menyimpan data mahasiswa.
- Pesanan menyimpan mahasiswa, menu, dan jumlah.
- MenuItem menyimpan data menu.

Diagram sesudah refactor



Alasan memakai inheritance

Makanan dan Minuman sama-sama merupakan MenuItem, sehingga data umum seperti kode, nama, harga, dan ketersediaan tetap disimpan di MenuItem. Makanan dan minuman kemudian memiliki data tambahan masing-masing.

Hasil Pengujian Inheritance

Setelah program dijalankan, object Makanan dan Minuman berhasil dibuat sebagai turunan dari MenuItem. Method info() yang dioverride juga berhasil menampilkan informasi tambahan sesuai jenis object.

Output yang diperoleh:

INHERITANCE

M01 - Nasi Goreng - Rp18000 - Tersedia: true - Jenis: Makanan Berat

M02 - Kopi Susu - Rp12000 - Tersedia: false - Ukuran: Large

Dari hasil tersebut terlihat bahwa Makanan menambahkan informasi jenis, sedangkan Minuman menambahkan informasi ukuran. Keduanya tetap menggunakan informasi umum dari MenuItem melalui super.info().

```
PS C:\Semester 3\PBO\Praktikum 4\Tugas Sebelumnya\kantin> java -cp . id.ac.polban.pbo.kantin.app.Main
=== DATA MAHASISWA ===
241001 - Asep
241002 - Siti

=== DATA MENU ===
Nasi Goreng tersedia: true
Kopi Susu tersedia: false
M01 - Nasi Goreng - Rp18000 - Tersedia: true - Jenis: Makanan Berat
M02 - Kopi Susu - Rp12000 - Tersedia: false - Ukuran: Large

=== DATA PESANAN ===
Pesanan #1 | Asep | Nasi Goreng | Jumlah: 2 | Total: Rp36000 | Dapat diproses: true
Pesanan #2 | Siti | Kopi Susu | Jumlah: 1 | Total: Rp12000 | Dapat diproses: false
Pesanan #3 | Asep | Nasi Goreng | Jumlah: 1 | Total: Rp18000 | Dapat diproses: true

=== STATIC ===
Nomor P1: 1
Nomor P2: 2
Nomor P3: 3
Jumlah pesanan dibuat: 3
```

tabel T4-01 sampai T4-07

ID	Pengujian	Hasil
T4-01	Constructor subclass	PASS
T4-02	Method warisan	PASS
T4-03	Method overriding	PASS
T4-04	super.method()	PASS
T4-05	Encapsulation	PASS
T4-06	Dua subclass	PASS
T4-07	Regression tugas sebelumnya	PASS